

React State Hands-On Lab — Solution

Building a Mall Entry/Exit Counter ("counterapp") Using React State

Objectives

- Explain React state

In this hands-on lab, you will learn how to:

- Use the React state object

Prerequisites

- Node.js
- NPM
- Visual Studio Code

Task

Create a React app named "counterapp" with a component named CountPeople that tracks how many people have entered and exited a mall. The component stores entryCount and exitCount in state, initialized in the constructor. It has two methods — updateEntry() and updateExit() — and two buttons: Login, which increments entryCount by 1, and Exit, which increments exitCount by 1. Both counts are displayed on the page.

Solution — Step by Step

Step 1: Create the React project "counterapp"

Open a terminal and run the following command to scaffold the project:

```
npx create-react-app counterapp
```

Once the setup finishes, navigate into the project folder:

```
cd counterapp
```

Step 2: Open the application in VS Code

```
code .
```

Step 3: Create the CountPeople component

Under the src folder, add a new file named CountPeople.js and start the class component:

```
import React, { Component } from 'react';

class CountPeople extends Component {
  // constructor, updateEntry(), updateExit()
  // and render() are added in the following steps
}
```

```
export default CountPeople;
```

Step 4: Initialize entryCount and exitCount using the constructor

Add a constructor that initializes both counts to zero in the component's state:

```
constructor(props) {  
  super(props);  
  this.state = {  
    entryCount: 0,  
    exitCount: 0  
  };  
  this.updateEntry = this.updateEntry.bind(this);  
  this.updateExit = this.updateExit.bind(this);  
}
```

Step 5: Implement updateEntry()

Add a method that increments entryCount by 1 whenever a person enters the mall:

```
updateEntry() {  
  this.setState(prevState => ({  
    entryCount: prevState.entryCount + 1  
  }));  
}
```

Step 6: Implement updateExit()

Add a method that increments exitCount by 1 whenever a person exits the mall:

```
updateExit() {  
  this.setState(prevState => ({  
    exitCount: prevState.exitCount + 1  
  }));  
}
```

Step 7: Implement render() with the Login and Exit buttons

Add the render() method to display both counts and two buttons: Login, which calls updateEntry(), and Exit, which calls updateExit():

```
render() {  
  return (  
    <div className="count-people">  
      <h1>Mall Visitor Counter</h1>  
      <p>People Entered: {this.state.entryCount}</p>  
      <p>People Exited: {this.state.exitCount}</p>  
      <button onClick={this.updateEntry}>Login</button>  
      <button onClick={this.updateExit}>Exit</button>  
    </div>  
  );  
}
```

```
}
```

Complete CountPeople.js

Putting all the pieces together, the complete CountPeople.js file looks like this:

```
import React, { Component } from 'react';

class CountPeople extends Component {
  constructor(props) {
    super(props);
    this.state = {
      entryCount: 0,
      exitCount: 0
    };
    this.updateEntry = this.updateEntry.bind(this);
    this.updateExit = this.updateExit.bind(this);
  }

  updateEntry() {
    this.setState(prevState => ({
      entryCount: prevState.entryCount + 1
    }));
  }

  updateExit() {
    this.setState(prevState => ({
      exitCount: prevState.exitCount + 1
    }));
  }

  render() {
    return (
      <div className="count-people">
        <h1>Mall Visitor Counter</h1>
        <p>People Entered: {this.state.entryCount}</p>
        <p>People Exited: {this.state.exitCount}</p>
        <button onClick={this.updateEntry}>Login</button>
        <button onClick={this.updateExit}>Exit</button>
      </div>
    );
  }
}

export default CountPeople;
```

Step 8: Add the CountPeople component to App.js

Update App.js to import and render the CountPeople component:

```
import React from 'react';
import './App.css';
import CountPeople from './CountPeople';

function App() {
  return (
    <div className="App">
      <CountPeople />
    </div>
  );
}

export default App;
```

Step 9: Build and run the application

In the command prompt, navigate into the counterapp folder (if not already there) and run:

```
cd counterapp
npm start
```

Open a browser window and type the following in the address bar:

```
http://localhost:3000
```

Every time the Login button is clicked, People Entered increases by 1. Every time the Exit button is clicked, People Exited increases by 1, confirming that the component's state is being updated and re-rendered correctly on each click.

Result

A React application named "counterapp" was successfully created with a CountPeople class component that stores entryCount and exitCount in state, initialized through the constructor. The updateEntry() and updateExit() methods update the respective counts using setState(), and are triggered by the Login and Exit buttons. The component was added to App.js and verified by running the application locally and confirming both counters increment correctly at http://localhost:3000.